

RARITONE BACKEND ARCHITECTURE & DEPLOYMENT RESEARCH DOCUMENTATION

Project Name: Raritone Virtual Try-On Platform

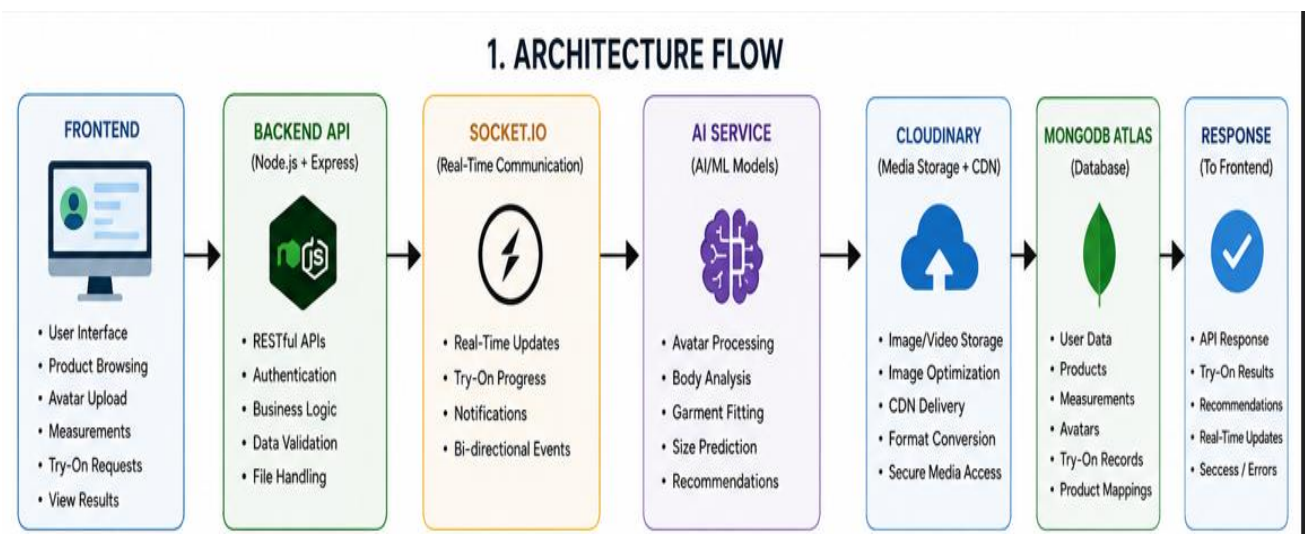
Prepared By: Nandini

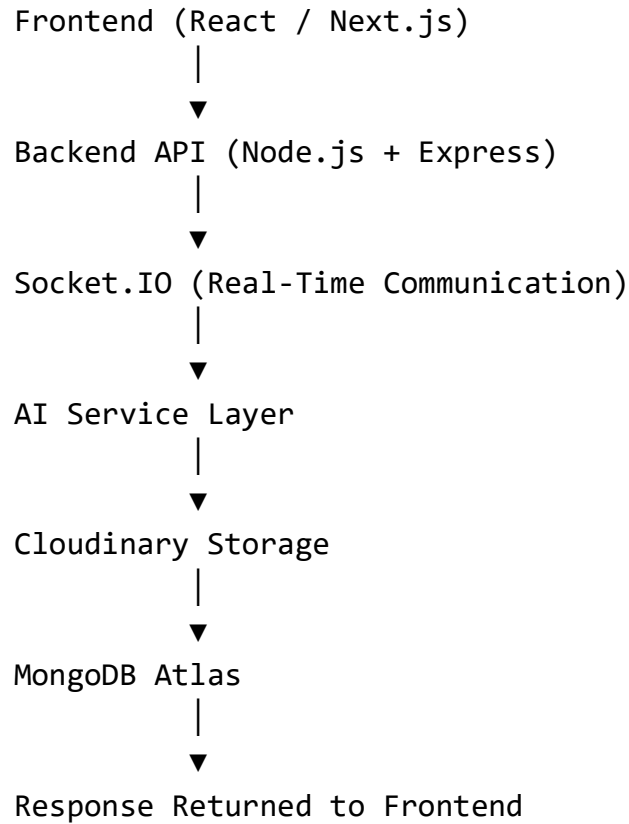
Module: Backend Architecture, Deployment & Monitoring Research

1. Backend Architecture Overview

The Raritone platform follows a modern scalable backend architecture designed to support virtual try-on functionality, real-time communication, AI processing, media management, and cloud-based deployment.

Architecture Flow





2. Architecture Component Explanation

Frontend Layer

Responsibilities

- User Authentication
- Product Browsing
- Avatar Upload
- Measurement Submission
- Virtual Try-On Requests
- Display AI Results

Technologies

- React.js
- Next.js
- Tailwind CSS

Backend API Layer

Responsibilities

- Route Handling
- Authentication & Authorization
- Business Logic
- Database Operations
- File Upload Management

Technologies

- Node.js
- Express.js
- JWT Authentication
- REST APIs

Socket.IO Layer

Purpose

Provides real-time communication between frontend and backend.

Use Cases

- AI processing status updates

- Live try-on progress
- Notifications
- Real-time order updates

Benefits

- Low latency
- Bi-directional communication
- Better user experience

AI Service Layer

Responsibilities

- Avatar Processing
- Body Measurement Analysis
- Garment Fitting
- Size Prediction
- Recommendation Generation

Future AI Modules

- Body Segmentation
- Pose Estimation
- Virtual Clothing Overlay
- Personalized Recommendations

Cloudinary Layer

Purpose

Cloud-based media storage and optimization.

Features

- Image Storage
- CDN Delivery
- Image Compression
- Automatic Resizing
- Format Conversion

Benefits

- Reduced server load
- Faster image delivery
- Better scalability

MongoDB Atlas Layer

Purpose

Primary cloud database.

Stores

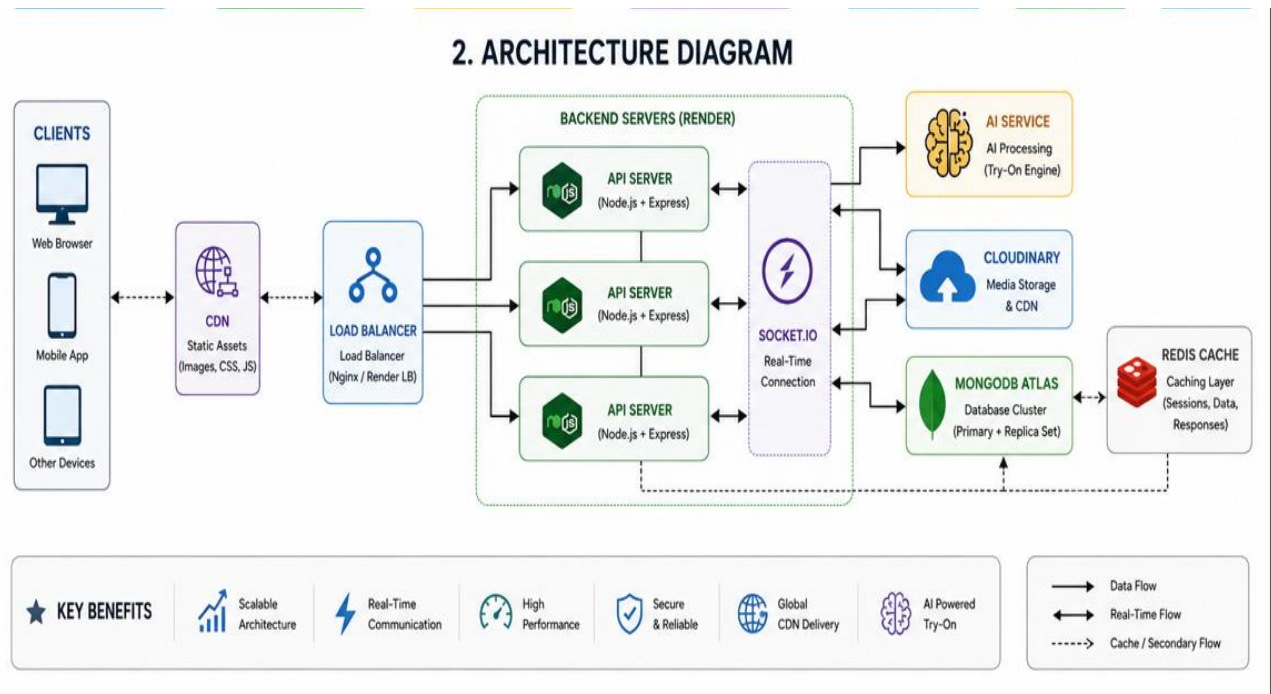
- Users
- Products
- Measurements
- Avatars
- TryOn Records
- Product Mappings

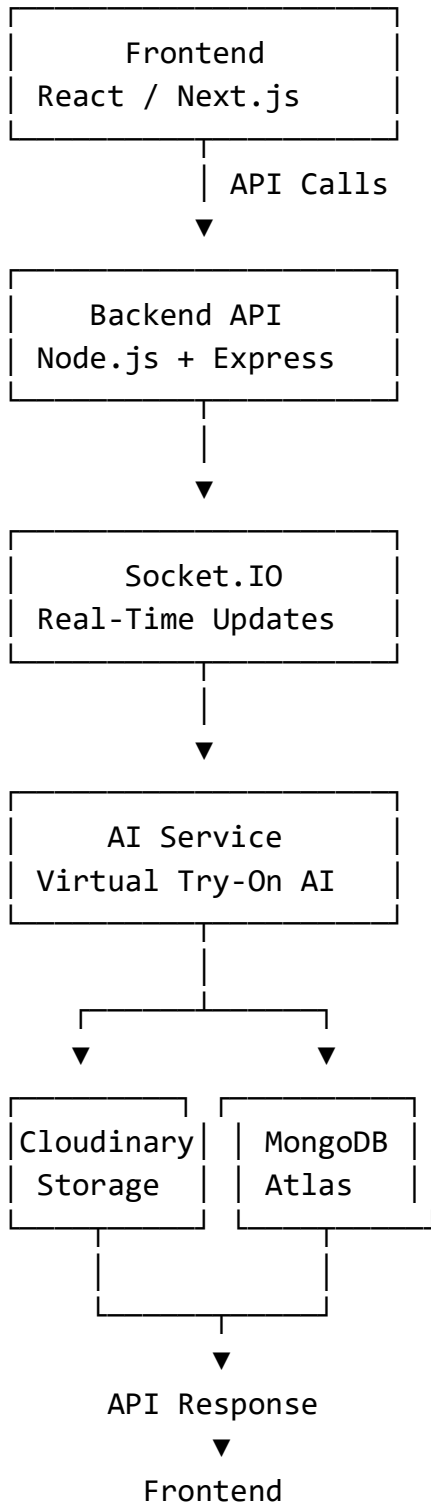
Benefits

- Managed Cloud Database
- Automatic Backups
- High Availability

- Global Scaling

3. Architecture Diagram





4. Deployment Strategy Research

Render Deployment

Overview

Render is a cloud hosting platform suitable for Node.js applications.

Benefits

- Easy deployment from GitHub
- Automatic builds
- SSL certificates
- Environment variable support
- Auto-scaling support

Deployment Process

1. Connect GitHub repository.
2. Configure build command.
3. Configure start command.
4. Add environment variables.
5. Deploy application.

MongoDB Atlas

Purpose

Managed MongoDB cloud database.

Features

- Database clustering
- Automatic backups

- Data replication
- Monitoring dashboard
- Security controls

Scaling Benefits

- Horizontal scaling
- Automatic failover
- Multi-region deployment

Redis Caching

Purpose

In-memory caching system.

Use Cases

- Product caching
- Session storage
- Frequently accessed data
- API response caching

Benefits

- Reduced database queries
- Faster response times
- Improved scalability

Load Balancing

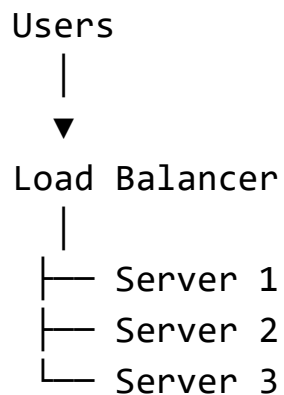
Purpose

Distributes traffic across multiple backend servers.

Benefits

- High availability
- Fault tolerance
- Better performance
- Traffic distribution

Example Flow



Microservices Architecture

Future Scaling Strategy

Instead of a single backend, separate services can be created:

Authentication Service

Product Service

Try-On Service

AI Service
Notification Service

Benefits

- Independent deployment
- Better scalability
- Easier maintenance
- Team productivity

CDN Scaling

Purpose

Content Delivery Network distributes media globally.

Benefits

- Faster image loading
- Reduced latency
- Lower backend load
- Global availability

Example

Cloudinary CDN serves images from locations nearest to users.

5. Monitoring Tools Research

Sentry

Purpose

Application error monitoring platform.

Features

- Error tracking
- Stack traces
- Performance monitoring
- Alert notifications

Benefits

- Faster debugging
- Production issue detection
- Improved reliability

Grafana

Purpose

Infrastructure monitoring and visualization platform.

Features

- Dashboards
- Metrics visualization
- Performance tracking
- System health monitoring

Monitored Metrics

- CPU usage
- Memory usage
- Request rates
- Database performance

Firestore Analytics

Purpose

User behavior analytics platform.

Tracks

- User engagement
- Session duration
- Feature usage
- Conversion rates

Benefits

- Product improvement insights
- User journey analysis
- Data-driven decisions

6. Progress Report

Completed Backend Features

Authentication Module

- ✓ User Registration
- ✓ User Login
- ✓ JWT Authentication
- ✓ Protected Routes

Product Module

- ✓ Product CRUD APIs
- ✓ Product Image Upload
- ✓ Product Management

Measurement Module

- ✓ Measurement Storage
- ✓ User Measurement APIs
- ✓ Measurement Retrieval

Avatar Module

- ✓ Avatar Upload APIs
- ✓ Avatar Management
- ✓ Cloudinary Integration

Try-On Module

- ✓ Database Design Completed
- ✓ API Structure Planned
- ✓ AI Workflow Planning Completed

Realtime APIs

Implemented / Planned

- ✓ Socket.IO Architecture Design
- ✓ Real-Time Event Planning
- ✓ Try-On Progress Updates
- ✓ Notification Workflow

Media Optimization

- ✓ Cloudinary Integration

✓ Image Compression Strategy

✓ CDN Delivery Planning

✓ Media Storage Optimization

AI Workflow Planning

✓ Avatar Processing Flow

✓ Product Mapping Design

✓ Size Recommendation Planning

✓ Virtual Try-On Pipeline Design

Future Scaling Preparation

✓ Render Deployment Strategy

✓ MongoDB Atlas Planning

✓ Redis Research

✓ Load Balancing Research

✓ Microservices Research

✓ Monitoring Stack Research

7. Conclusion

The backend architecture is designed to support a scalable AI-powered virtual try-on platform. The system integrates Node.js, Express, Socket.IO, Cloudinary, MongoDB Atlas, and future AI services to provide real-time user experiences and efficient media processing.

The deployment strategy incorporates Render hosting, MongoDB Atlas, Redis caching, load balancing, microservices architecture, and CDN optimization. Monitoring tools such as Sentry, Grafana, and Firebase Analytics ensure application reliability, performance tracking, and user behavior analysis.

The project is prepared for future scaling while maintaining maintainability, performance, and cloud-native architecture principles.